COS 122 Exam Opportunity 3 Formula and Method Sheet

Chapters 9, 11 and 12 · 2026

Contents

1. Chapter 9: Uniprocessor Scheduling 2
   - Types of scheduling 2
   - Scheduling criteria 2
   - Symbols, measures and decision mode 2
   - Drawing a time-scale diagram 3
   - FCFS (first come, first served) 3
   - Round robin (RR) with quantum q 3
   - SPN (shortest process next) 4
   - SRT (shortest remaining time) 4
   - Turnaround, normalised turnaround and waiting time 5
   - HRRN (highest response ratio next) 5
   - Feedback (multilevel feedback queues) 6
   - Priority scheduling 7
   - Naming the policy from a given schedule 7
2. Chapter 11: I/O Management and Disk Scheduling 8
   - I/O devices 8
   - Techniques for performing I/O 8
   - Buffering 8
   - Disk access time 9
   - Disk scheduling 10
   - RAID 12
   - Disk cache 13
3. Chapter 12: File Management 14
   - Operations on files 14
   - File organisations 14
   - Record blocking 15
   - B-trees 15
   - File allocation 17
   - Free space management 18
   - UNIX inodes 18
   - Directories, access rights and sharing 19

1. Chapter 9: Uniprocessor Scheduling

Types of scheduling; criteria; FCFS, RR, SPN, SRT, HRRN, Feedback and Priority on a time-scale diagram; turnaround and waiting time; naming a policy from a schedule

Types of scheduling

A **process** is a program in execution. The **ready queue** holds the processes that are ready to run and are waiting for the processor. The **degree of multiprogramming** is the number of processes in main memory at one time. **Swapping** moves a process out of main memory to disk (swap out) and back again (swap in).

Type	The decision it makes	When it runs
Long-term	whether to add a new process to the pool of processes to be executed (controls the degree of multiprogramming: *when* to admit and *which* job)	rarely: when a job is submitted or a process ends
Medium-term	whether to add a process to those partly or fully in main memory (part of swapping: swap in or swap out)	when memory must be managed
Short-term (the **dispatcher**: the part of the operating system that gives the processor to a process)	which ready process the processor runs next	frequently, at events: clock interrupts, I/O interrupts, operating-system calls, signals (for example semaphores)
I/O	which process's pending I/O request an available I/O device handles	when a device becomes free

Scheduling criteria

Criteria are sorted two ways: **user-oriented** (what one user or process sees) or **system-oriented** (how well the whole system is used), and **performance-related** (measurable: a time or a rate) or **not performance-related** (a quality that is hard to measure).

	Performance-related	Not performance-related (other)
User-oriented	turnaround time; response time; deadlines	predictability
System-oriented	throughput; processor utilisation	fairness; enforcing priorities; balancing resources

1. Find the row (user or system) and the column (performance or other) the question names.
2. Write one name from that cell. "Name (not describe)" means the name alone earns the mark.

Exam question: EO3 2024, question 1

(a) Name one criterion that is system-oriented and non-performance-related: **fairness** (or enforcing priorities, or balancing resources). (b) System-oriented and performance-related: **throughput** (or processor utilisation). (c) User-oriented and performance-related: **response time** (or turnaround time, or deadlines).

Common mistake

"Starvation" is not a criterion: it is a risk of some policies (SPN, SRT, Feedback, Priority). It scored 0 for question 1(a) in 2024.

Symbols, measures and decision mode

Symbol	Meaning (unit: the time unit of the question, for example ms)
arrival	the time the process enters the ready queue
s or T_s	service time: the total processor time the process needs
w	time spent waiting so far (now $-$ arrival, while it has not run)
e	time spent executing so far; $s - e$ is the remaining time
finish	the time the last unit of the process ends
T_r	turnaround time $=$ finish $-$ arrival
T_r/T_s	normalised turnaround (no unit; 1.00 means no waiting at all)
waiting time	$T_r - T_s$ = finish $-$ arrival $-$ service

Non-preemptive: a running process keeps the processor until it ends or blocks. **Preemptive**: the operating system may stop it and put it back in the ready queue. This happens at a new arrival, at an interrupt, or at the end of a **time**

quantum q (the longest time a process may run in one turn). **Overhead** is processor time spent on scheduling and switching between processes instead of on the processes. Preemption costs more overhead, but it stops one process from holding the processor for a long time.

Policy	Selection function	Decision mode	Effect on processes	Starvation
FCFS	$\max[w]$	non-preemptive	penalises short and I/O-bound processes	no
RR	constant	preemptive (at time quantum)	fair treatment	no
SPN	$\min[s]$	non-preemptive	penalises long processes	possible
SRT	$\min[s-e]$	preemptive (at arrival)	penalises long processes	possible
HRRN	$\max\left[\frac{w+s}{s}\right]$	non-preemptive	good balance	no
Feedback	see Feedback below	preemptive (at time quantum)	may favour I/O-bound processes	possible

Read the selection function as the rule for choosing the next process:

- $\max[w]$: the process that has waited longest (w = time waited so far).
- constant: no ranking; the processes take turns.
- $\min[s]$: the smallest service time. $\min[s-e]$: the smallest remaining time.
- $\max[(w+s)/s]$: the largest response ratio (see HRRN below).

An **I/O-bound** process uses the processor in short bursts and spends most of its time waiting for I/O. **Starvation**: a process waits for ever because other processes are always chosen before it. Throughput: high for SPN, SRT and HRRN. RR may have low throughput if the quantum is too small. Overhead: lowest for FCFS and RR. It can be high for SPN, SRT, HRRN and Feedback.

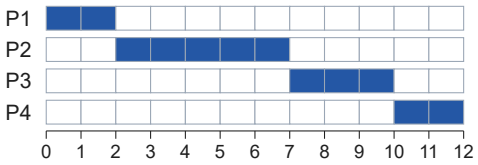
Drawing a time-scale diagram

1. Draw one box per time unit, numbered from 0. A row strip (one row) or one row per process (as below) are both accepted.
2. A process that arrives at time t is in the ready queue for the decision made at time t.
3. At each decision point apply the policy's selection function to the ready processes only.
4. Fill one box per unit with the running process until every process has received its full s. Check: the number of boxes for each process equals its service time.

FCFS (first come, first served)

1. Order the processes by arrival time.
2. Run each one to completion in that order. If the processor is free and no process has arrived, it is idle.

Example (the EO3 2024 workload: P1 arrives 0, s = 2; P2 1, 5; P3 3, 3; P4 6, 2):



FCFS: P1 0–2, P2 2–7, P3 7–10, P4 10–12.

Common mistake

FCFS never splits a process: every process appears as one solid block. If any process is split, the policy is not FCFS.

Round robin (RR) with quantum q

Tie rule used in this course

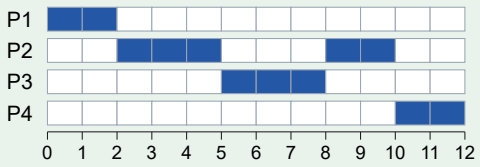
A new process may arrive at the same moment that a quantum ends. Then the **new process joins the back of the ready queue first**. The process that was stopped joins behind it. This is the order in the course textbook by William Stallings (Figure 9.5 and Table 9.5, RR $q=1$). Tutorial 8, question 2 ($q=2$) uses it too: at $t=6$ P4 arrives as P1 is stopped, and P4 runs before P1.

1. Keep a FIFO ready queue. At $t = 0$ start the first arrival.
2. The running process runs for q units, or less if it finishes first.
3. At the end of the quantum: first add every process that arrived up to and including this instant to the back of the queue, then put the pre-empted process at the back.
4. Run the front process. If the queue is empty, the same process keeps running with a new quantum.
5. A process that finishes inside its quantum gives up the processor at once; the next process gets a full new quantum.

Exam question: EO3 2024, question 2(d)

Show the schedule using RR with $q = 3$ for P1 (arrives 0, $s = 2$), P2 (1, 5), P3 (3, 3), P4 (6, 2).

$t = 0$: only P1; it runs 0–2 and finishes inside its quantum.
$t = 2$: queue P2 (P3 not yet arrived). P2 runs 2–5 (3 units, 2 left). P3 arrived at 3.
$t = 5$: queue P3, P2. P3 runs 5–8 and finishes. P4 arrives at 6 and joins behind P2.
$t = 8$: queue P2, P4. P2 runs 8–10 (its last 2 units).
$t = 10$: P4 runs 10–12.
Answer: **P1 P1 P2 P2 P2 P3 P3 P3 P2 P2 P4 P4**.



Question	Workload (arrival, s)	RR $q = 2$ schedule
EO3 2023, question 1(d)	P1 0, 2; P2 1, 6; P3 3, 3; P4 5, 1	P1 P1 P2 P2 P3 P3 P2 P2 P4 P3 P2 P2
Mock test, question 1(d)	P1 0, 5; P2 2, 2; P3 3, 5; P4 5, 1	P1 P1 P2 P2 P1 P1 P3 P3 P4 P1 P3 P3 P3

Effect of the quantum (Tutorial 8, question 2)

P1 0, 10; P2 1, 6; P3 3, 8; P4 6, 4. Average T_r: $q = 2$: 20.50; $q = 5$: 21.75; $q = 10$: 17.00. The average does not change steadily with q. If q is at least the longest service time, RR becomes FCFS. A small q gives short processes a fast response but causes many process switches (overhead). Choose q a little larger than a typical interaction. With the opposite tie rule (stopped process first) the $q = 2$ average would be 21.00, which is not the tutorial's answer.

SPN (shortest process next)

1. Non-preemptive: a running process always finishes.
2. When the processor becomes free, list the processes that have arrived and not run.
3. Choose the smallest service time s. Tie: the earlier arrival. *(Assumed)*

Exam question: EO3 2024, question 2(b)

Same workload. Name a policy that gives P1 P1 P2 P2 P2 P2 P2 P4 P4 P3 P3 P3.

No process is split, so the policy is non-preemptive.
$t = 2$: P1 ends; only P2 has arrived. P2 runs 2–7.
$t = 7$: P3 ($s = 3$, arrived 3) and P4 ($s = 2$, arrived 6) wait. P4 runs first although P3 arrived earlier, so the order is not FCFS. P4 has the smaller s.
$t = 9$: P4 ends; P3 is the only one left, runs 9–12.
Policy: **SPN**. Check: running SPN on the workload gives every box of the schedule.

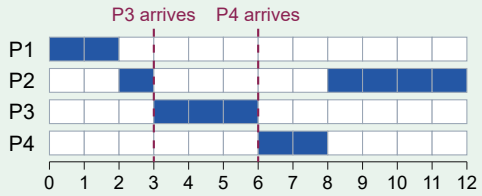
SRT (shortest remaining time)

1. Preemptive at arrival: make a decision whenever a process arrives or the running process finishes.
2. Compare the remaining time $s - e$ of the running process with the service time of each newcomer and the remaining times of the waiting processes.
3. Run the smallest remaining time. A newcomer takes over **only if its time is strictly smaller**; on a tie the running process continues (EO3 2023, question 1(c), at $t = 5$).

Exam question: EO3 2024, question 2(c)

Same workload. Name a policy that gives P1 P1 P2 P3 P3 P3 P4 P4 P2 P2 P2 P2.

$t = 2$: P2 starts (remaining 5).
$t = 3$: P3 arrives needing 3 $<$ P2's 4 left: P3 takes over, runs 3–6.
$t = 6$: P3 ends; P4 arrives needing 2 $<$ P2's 4: P4 runs 6–8.
$t = 8$: P2 finishes 8–12.
P2 is split, so the policy is preemptive. The blocks have different lengths (2, 1, 3, 2, 4), so it is not RR. At every arrival and every finish the process with the least time left runs: **SRT**.



Turnaround, normalised turnaround and waiting time

1. From the finished diagram read each finish time (the right edge of the process's last box).
2. T_r = finish − arrival; T_r/T_s to two decimals; waiting $= T_r - T_s$.
3. Mean = sum ÷ number of processes. Round only the final value.

Worked row (P2 in the SRT schedule above): P2 arrives at 1 and its last box ends at 12. $T_r = 12 - 1 = 11$; $T_r/T_s = 11/5 = 2.20$; waiting $= 11 - 5 = 6$.

Example: the SRT schedule above.

	Arr	T_s	Fin	T_r	T_r/T_s	Wait
P1	0	2	2	2	1.00	0
P2	1	5	12	11	2.20	6
P3	3	3	6	3	1.00	0
P4	6	2	8	2	1.00	0
Mean				4.50	1.30	1.50

Tutorial 8, question 1 (P1 0, 60, priority 4; P2 5, 30, 2; P3 15, 90, 1; P4 25, 20, 3; ms):

- SRT: P1 0–5, P2 5–35, P4 35–55, P1 55–110, P3 110–200; average waiting 38.75 ms.
- Non-preemptive priority: P1 0–60, P3 60–150, P2 150–180, P4 180–200; average waiting 86.25 ms.
- RR $q = 20$: P1 0–20, P2 20–40, P3 40–60, P1 60–80, P4 80–100, P2 100–110, P3 110–130, P1 130–150, P3 150–200; average waiting 78.75 ms.

Mock test, question 3 (SRT; P1 0, 7; P2 2, 4; P3 3, 1; P4 5, 3; P5 6, 2):

- Schedule: P1 P1 P2 P3 P2 P2 P2 P5 P5 P4 P4 P4 P1 P1 P1 P1 P1.
- Mean T_r 6.60; mean T_r/T_s 1.70; mean waiting 3.20.

HRRN (highest response ratio next)

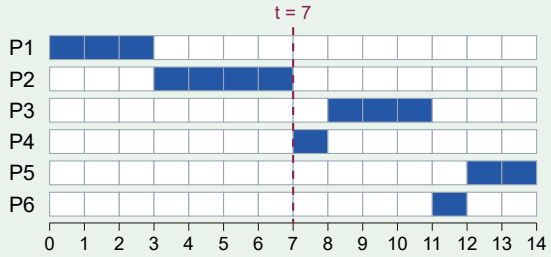
$$R = \frac{w + s}{s} \qquad w = \text{time waited so far} = t - \text{arrival}, \quad s = \text{service time}$$

w and s must be in the same time unit. R is a ratio, so it has no unit.

1. Non-preemptive: decide only when the running process finishes, at time t.
2. For each process that has arrived and not run: $w = t -$ arrival, then $R = (w + s)/s$, to two decimals. A process arriving at t has $w = 0$, so $R = 1$.
3. Run the largest R. Short processes and long waiters both get a high R, so nothing starves.

Exam question: EO3 2024, question 3

Workload P1 0, 3; P2 2, 4; P3 3, 3; P4 5, 1; P5 6, 2; P6 7, 1. The schedule after 7 steps is P1 P1 P1 P2 P2 P2 P2.
(a) Response ratio of each active process at time 7 (two decimals):
P3: $w = 7 - 3 = 4$ $R = (4 + 3)/3 = \mathbf{2.33}$ P4: $w = 7 - 5 = 2$ $R = (2 + 1)/1 = \mathbf{3.00}$
P5: $w = 7 - 6 = 1$ $R = (1 + 2)/2 = \mathbf{1.50}$ P6: $w = 7 - 7 = 0$ $R = (0 + 1)/1 = \mathbf{1.00}$
Next to run: the largest ratio, **P4**. The full schedule: P1 P1 P1 P2 P2 P2 P2 P4 P3 P3 P3 P6 P5 P5.



HRRN, EO3 2024 question 3 workload. The dashed line is the decision at $t = 7$.

Question	Working and answer
EO3 2023, question 2(b): P1 0, 3; P2 2, 4; P3 4, 1; P4 5, 2; decision at $t = 7$	$R(\text{P3}) = (3 + 1)/1 = 4.00$; $R(\text{P4}) = (2 + 2)/2 = 2.00$. Next: P3.
Mock test, question 2(b): decision at $t = 5$	R(P2) = 2.33, R(P3) = 2.50, R(P4) = 2.00. Next: P3.
Tutorial 8, question 3 (ms)	Schedule P1 0–40, P2 40–90, P4 90–120, P3 120–200, P5 200–270. Ratios at 40: P2 1.60, P3 1.25, P4 1.00 ⇒ P2. At 90: P3 1.88, P4 2.67, P5 1.43 ⇒ P4. At 120: P3 2.25, P5 1.86 ⇒ P3. Average waiting 64 ms.

Feedback (multilevel feedback queues)

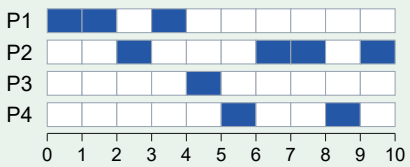
Feedback rules (Stallings)

- Ready queues RQ0 (highest), RQ1, RQ2, …; each queue is FCFS. A new process joins the back of RQ0.
- The scheduler runs the front process of the highest non-empty queue.
- Preemption happens only when a quantum ends; an arrival in the middle of a quantum waits.
- A process that uses its whole quantum moves down one queue, **if another process is waiting**. If nothing else is ready it stays in its queue and gets a new quantum.
- $q = 1$: every queue has quantum 1. $q = 2^i$: RQi has quantum 2^i (1, 2, 4, 8, …).
- The textbook has no limit on queues. When a question fixes the number of queues, follow its rule for the lowest queue (Tutorial 8, question 4: the lowest queue is FCFS).

1. At each quantum boundary add the arrivals to RQ0, then place the pre-empted process one queue lower.
2. Pick the front of the highest non-empty queue; give it the quantum of that queue.
3. Repeat until every process has its full service time.

Exam question: EO3 2023, question 2(a): Feedback with $q = 2^i$

P1 0, 3; P2 2, 4; P3 4, 1; P4 5, 2.
0–1 P1 (RQ0, $q = 1$); alone, so it stays in RQ0: 1–2 P1.
At 2 P2 arrives: P1 drops to RQ1. 2–3 P2 (RQ0), then P2 drops to RQ1.
3–4 P1 (RQ1, $q = 2$, needs 1): P1 ends. 4–5 P3 ends.
5–6 P4 (RQ0), drops.
6–8 P2 (RQ1, $q = 2$), drops to RQ2. 8–9 P4 ends. 9–10 P2 ends.
Answer: **P1 P1 P2 P1 P3 P4 P2 P2 P4 P2**.



With $q = 1$ the same workload gives P1 P1 P2 P1 P3 P4 P2 P4 P2 P2.

$q = 1$ compared with $q = 2^i$. With $q = 1$ a long process is stopped after every unit. It sinks to the low queues quickly, so it may wait a long time or starve. With $q = 2^i$ the quantum doubles at each level. A long process gets longer runs as it drops, so it is stopped less often and its turnaround is shorter.

Mock test, question 2(a) (Feedback $q = 2^i$; P1 0, 5; P2 1, 3; P3 2, 2; P4 4, 1):

- 0–1 P1 (RQ0, $q = 1$). P2 arrives at 1, so P1 drops to RQ1.
- 1–2 P2 (RQ0), drops to RQ1. 2–3 P3 (RQ0), drops to RQ1.
- 3–5 P1 (RQ1, $q = 2$). P4 arrives at 4 and waits for the quantum to end. P1 drops to RQ2 (2 units left).
- 5–6 P4 (RQ0) ends. 6–8 P2 (RQ1, $q = 2$) ends. 8–9 P3 (RQ1) ends. 9–11 P1 (RQ2, $q = 4$) ends.
- Answer: P1 P2 P3 P1 P1 P4 P2 P2 P3 P1 P1.

Tutorial 8, question 4 (3 queues, $q = 20$, lowest queue FCFS): P1 0–20, P2 20–40, P3 40–60, P4 60–80, P5 80–100, P1 100–120, P2 120–140, P3 140–160, P4 160–170, P5 170–190, P2 190–200, P3 200–240, P5 240–270; average waiting 120 ms.

Priority scheduling

1. A lower number means a higher priority (Tutorial 8, question 1; Stallings Problem 9.1).
2. Non-preemptive: when the processor is free, run the highest-priority process that has arrived; it runs to the end.
3. Preemptive: at every arrival, a newcomer takes over only if its priority is strictly higher; the pre-empted process waits in the ready set.
4. Tie on priority: the earlier arrival. *(Assumed)*

Mock test, question 4 (arrival, s, priority: P1 0, 4, 3; P2 1, 3, 2; P3 3, 2, 1; P4 4, 3, 4). Non-preemptive: P1 P1 P1 P1 P3 P3 P2 P2 P2 P4 P4 P4. Preemptive:

- 0–1 P1 (only process).
- $t = 1$: P2 (priority 2) is higher than P1 (3): P2 takes over, runs 1–3.
- $t = 3$: P3 (priority 1) is higher than P2: P3 runs 3–5 and ends. P4 (priority 4) arrives at 4 and does not take over.
- $t = 5$: waiting P2 (2), P1 (3), P4 (4). P2 runs 5–6 and ends; P1 runs 6–9; P4 runs 9–12.
- Answer: P1 P2 P2 P3 P3 P2 P1 P1 P1 P4 P4 P4.

Starvation: a low-priority process (here P4) may never run while higher-priority processes keep arriving. The remedy is **ageing**: raise a process's priority the longer it waits.

Naming the policy from a given schedule

1. Look for a **split** process (one that appears, stops, then appears again). No split: FCFS, SPN, HRRN or non-preemptive priority. Split: RR, SRT, Feedback or preemptive priority.
2. No split: check the decision points. Arrival order kept ⇒ FCFS. A shorter process jumps an earlier arrival ⇒ SPN. A choice that fits neither: compute the HRRN ratios.
3. Split: blocks of equal length q taking turns ⇒ RR. A newcomer with less time takes over at its arrival ⇒ SRT. Newcomers run after a quantum of 1 and long processes get blocks of 1, 2, 4 ⇒ Feedback.
4. Check your answer by running the policy on the workload: it must produce every box.

Source	Schedule	Policies that produce it
EO3 2024, 2(a)	P1 P1 P2 P2 P2 P2 P2 P3 P3 P3 P4 P4	FCFS, HRRN
EO3 2024, 2(b)	P1 P1 P2 P2 P2 P2 P2 P4 P4 P3 P3 P3	SPN
EO3 2024, 2(c)	P1 P1 P2 P3 P3 P3 P4 P4 P2 P2 P2 P2	SRT
EO3 2023, 1(a)	P1 P1 P2×6 P3 P3 P3 P4	FCFS
EO3 2023, 1(b)	P1 P1 P2×6 P4 P3 P3 P3	SPN, HRRN
EO3 2023, 1(c)	P1 P1 P2 P3 P3 P3 P4 P2×5	SRT
Mock test, 1(a)	P1×5 P4 P2 P2 P3×5	SPN
Mock test, 1(b)	P1 P1 P2 P2 P1 P4 P1 P1 P3×5	SRT
Mock test, 1(c)	P1×5 P2 P2 P3×5 P4	FCFS

When two policies produce the same schedule, write the policy with the simpler rule (FCFS for EO3 2024, question 2(a); SPN for EO3 2023, question 1(b)). *(Assumed)* The credited answers on the marked scripts are FCFS and SPN; the HRRN entry in the table only records that HRRN gives the same boxes.

2. Chapter 11: I/O Management and Disk Scheduling

I/O devices and techniques; buffering; disk access time; FIFO, SSTF, SCAN and C-SCAN with and without LOOK; RAID; disk cache

I/O devices

Three categories

- **Human readable**: talk to the user (printer, terminal, display, keyboard, mouse).
- **Machine readable**: talk to equipment (disk drive, USB key, sensor, controller, actuator).
- **Communication**: talk to remote devices (digital line driver, modem).

Six ways devices differ: data rate; application; complexity of control; unit of transfer (block or stream of bytes); data representation (encoding); error conditions.

Block-oriented device: stores data in fixed-size blocks and transfers one block at a time; a block can be addressed by its number (disk, USB key).
Stream-oriented device: transfers data as a stream of bytes with no block structure (terminal, printer, communication port, mouse).

Techniques for performing I/O

	No interrupts	Use of interrupts
I/O-to-memory transfer through the processor	programmed I/O	interrupt-driven I/O
Direct I/O-to-memory transfer	–	direct memory access (DMA)

- **Programmed I/O**: the processor issues the command, then keeps checking the module's status (busy waiting) until the operation is complete.
- **Interrupt-driven I/O**: the processor issues the command and continues with other work; the module interrupts it when the data is ready. Non-blocking: the process continues; blocking: the process is suspended and another runs.
- **DMA**: a DMA module transfers a whole block between the device and memory without the processor. The processor sends it four items: read or write; the address of the I/O device; the starting memory location; the number of words. The DMA module interrupts once, when the whole block is done.

Buffering

Why: a process waiting for I/O cannot be swapped out, and transfers are faster if input is read ahead and output is written later. Buffering smooths peaks in demand; it cannot keep up if the process always needs data faster than the device delivers it.

- **No buffer**: the device transfers straight to the process.
- **Single buffer**: the OS reads the next block into a system buffer while the process works on the current one (reading ahead).
- **Double buffer** (buffer swapping): two system buffers; the process empties one while the device fills the other.
- **Circular buffer**: more than two buffers, used in turn, for bursts.

Scheme	Time per block
No buffer	$T + C$
Single buffer	$\max[C, T] + M$
Double buffer	$\max[C, T]$

T = time to input one block, C = processing (computation) time per block, M = time to move the block from the system buffer to user space (all in ms). On this page T is the input time; in disk access below, T is the transfer time.

- No buffer: the process waits for the input (T), then works on the block (C).
- Single: the next block is read while the process works on this one, so a block costs the longer of C and T; then the block is moved to the process (M).
- Double: the process takes the block from one buffer while the other fills, so no move time is added.

Exam question: Mock test, question 4

$T = 5$, $C = 3$, $M = 1$ ms. No buffer $5 + 3 = 8$ ms; single $\max[3, 5] + 1 = 6$ ms; double $\max[3, 5] = 5$ ms.

Disk access time

A disk has one or more platters; each platter has two **surfaces**. A surface is divided into rings called **tracks**, and each track into **sectors** (the unit read or written). To read data the arm first moves the head to the right track (the **seek**). Then the disk turns until the sector is under the head (rotational delay). Then the data is read as it passes (transfer).

Symbol	Meaning	Unit
T_s	average seek time: move the arm to the track (start-up, traverse, settle). Not the service time of Chapter 9	ms (given)
rpm	rotation speed	revolutions per minute
r	rotation speed = rpm $\div 60$	revolutions per second
$1/r$	time for one revolution	s; $\times 1000$ for ms
R_d	average rotational delay = $\frac{1}{2r}$ (half a revolution)	s; $\times 1000$ for ms
N	bytes on one track = bytes per sector $\times$ sectors per track	bytes
b	number of bytes to transfer	bytes
T	transfer time = $\frac{b}{rN}$	s; $\times 1000$ for ms
k	number of tracks the file fills = b/N	tracks
T_a	total access time	ms

$$T_a = T_s + \frac{1}{2r} + \frac{b}{rN} \quad \text{(one request)} \qquad T_a = T_s + k \cdot \frac{1}{2r} + \frac{b}{rN} \quad \text{(a sequential file on } k = b/N \text{ tracks)}$$

Units: 1 s = 1000 ms. MB in these questions is 2^{20} = 1,048,576 bytes and KB is 2^{10} = 1024 bytes; always use the byte count printed in the question.

Steps for reading a file stored sequentially ("include all rotational delay")

1. r = rpm$/60$ rev/s; one revolution = $1000/r$ ms; R_d = half of that.
2. N = bytes/sector $\times$ sectors/track.
3. Tracks $k = b/N$ (round up if the last track is only partly used *(Assumed)*).
4. $T = b/(rN)$ s, times 1000 for ms. Reading one full track takes one revolution, so T is also k revolutions.
5. $T_a = T_s + k \times R_d + T$: one seek (adjacent tracks need no further seek), one rotational delay for every track, then the transfer.
6. Round only the final answer to two decimals.

Exam question: EO3 2024, question 4(a)

1024 bytes/sector, 256 sectors/track, 1200 tracks/surface, 2 surfaces, 15,000 rpm, T_s = 4 ms, a 1 MB (1,048,576 bytes) file stored sequentially. Total time T_a (include all rotational delay), in ms to two decimals?

$r = 15000/60 = 250$ rev/s; one revolution = $1000/250 = 4$ ms; $R_d = 2$ ms.
$N = 1024 \times 256 = 262{,}144$ bytes; $k = 1{,}048{,}576/262{,}144 = 4$ tracks.
$T = 1{,}048{,}576/(250 \times 262{,}144)$ s $= 0.016$ s $= 16$ ms.
$T_a = 4 + 4 \times 2 + 16 =$ **28.00 ms**. The tracks per surface and the number of surfaces are not needed.

Common mistake

Using one rotational delay for the whole file gives $4 + 2 + 16 = 22.00$ ms; this answer scored 0 in the 2024 paper. The words "include all rotational delay" mean one R_d per track.

Question	Working and answer
Tutorial 9, question 2: 512 B/sector, 512 sectors/track, 12,000 rpm, T_s = 4.5 ms, 1,572,864 bytes	$r = 200$ rev/s; $R_d = 2.50$ ms; $N = 262{,}144$ bytes; $k = 6$; $T = 30$ ms. $T_a = 4.5 + 6 \times 2.50 + 30 =$ **49.50 ms**.
EO3 2023, question 3(a): 4096 B/sector, 512 sectors/track, 7200 rpm, T_s = 4 ms, 4,194,304 bytes	$r = 120$ rev/s; one revolution 8.33 ms; $R_d = 4.17$ ms; $k = 2$; $T = 16.67$ ms. $T_a = 4 + 2 \times R_d + T =$ **29.00 ms** (keep exact values until the end). An answer of 24.85 pays only one R_d (exact value 24.83). By the 2024 marking, that answer loses the mark.

Steps for a file scattered at random (each sector read separately)

1. Number of sectors = file bytes $\div$ bytes per sector.
2. Time per sector = $T_s + R_d$ + (one revolution $\div$ sectors per track). The last term is the time for one sector to pass under the head.
3. Total = number of sectors $\times$ time per sector.

Mock test, question 2 (1024 B/sector, 512 sectors/track, 15,000 rpm, T_s = 3 ms, 2,097,152 bytes):

- (a) $r = 15000/60 = 250$ rev/s; one revolution $= 4$ ms; R_d = **2.00 ms**.
- (b) Sequential: $k = 4$ tracks; $T = 16$ ms; $T_a = 3 + 4 \times 2 + 16 =$ **27.00 ms**.
- (c) Random: sectors $= 2{,}097{,}152/1024 = 2{,}048$. Per sector $3 + 2 + 4/512 = 5.0078$ ms. Total $2{,}048 \times 5.0078 =$ **10256.00 ms**.

Sequential storage takes much less time: the seek is paid once and the rotational delay once per track, not once per sector.

Disk scheduling

Rules used in the course

- With n tracks, they are numbered 0 to $n - 1$ (200 tracks: 0 to 199).
- **Direction comes from the previous position**: "moved from 119 to 120" means moving up (towards higher numbers); "at 110, previous request 135" means moving down (towards 0).
- Every arm movement counts, including the end-to-end return of C-SCAN.
- Average seek length = total tracks traversed ÷ number of requests.
- "SCAN" and "C-SCAN" alone mean the LOOK versions; use the version the question names.

Policy	Steps for choosing the next track
FIFO	serve the requests in arrival order.
SSTF (shortest service time first)	serve the request nearest to the current position, in either direction. Tie: keep the current direction. *(Assumed)*
SCAN (with LOOK)	Keep moving in the current direction and serve the requests in track order. At the last request ahead, turn and serve the rest.
SCAN (no LOOK)	Keep moving in the current direction and serve the requests in track order. Go on to the end track (0 or $n - 1$), then turn. Serve the rest and stop at the last request.
C-SCAN (with LOOK)	Serve requests in one direction only. After the last request ahead, jump to the farthest request at the other end (the jump counts). Serve the rest in the same direction.
C-SCAN (no LOOK)	Serve requests in one direction only. Go on to the end track, then return to the opposite end track (both legs count). Serve the rest in the same direction.

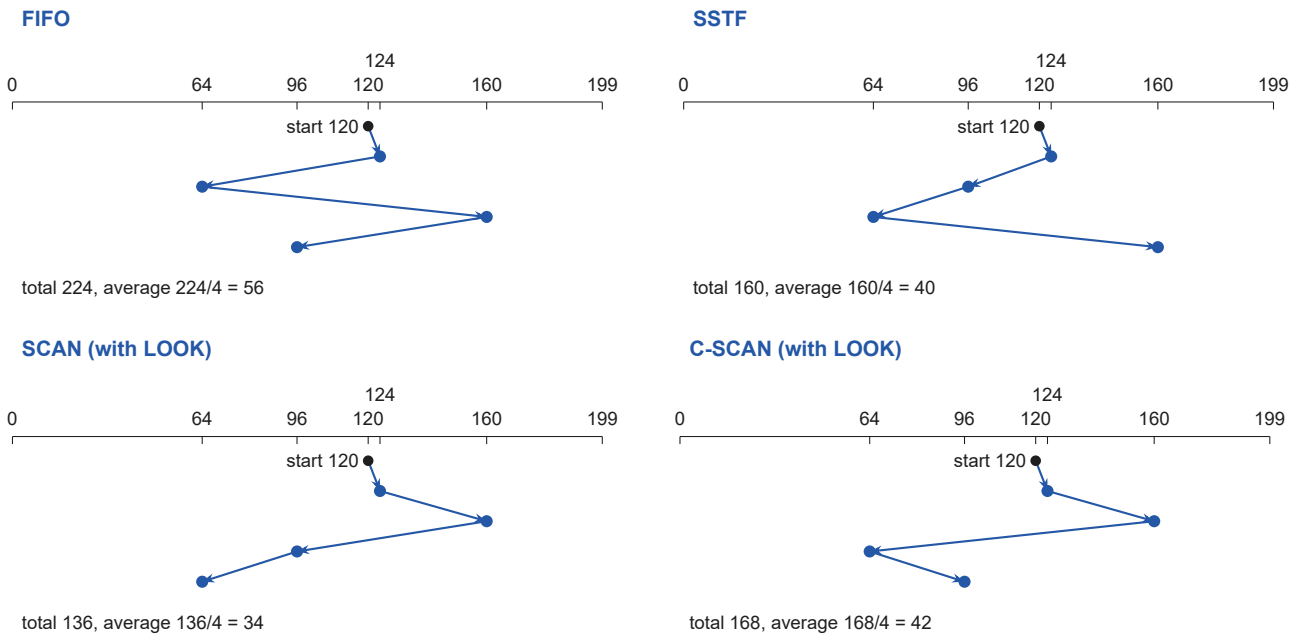
1. Write the start track and the direction.
2. Find the service order with the policy's rule.
3. For each request, tracks traversed = |previous stop − this track|; if the arm turned at an end track or jumped on the way, add every leg to this request's row.
4. Add the column; divide by the number of requests.

Exam question: EO3 2024, question 5(a)

200 tracks (0 to 199). The head has just moved from track 119 to track 120 (so it is moving up). Requests in arrival order: 124, 64, 160, 96. For each algorithm give the order, the tracks traversed for each request and the average seek length.

(i) FIFO		(ii) SSTF		(iii) SCAN (LOOK)		(iv) C-SCAN (LOOK)	
next	tracks	next	tracks	next	tracks	next	tracks
124	4	124	4	124	4	124	4
64	60	96	28	160	36	160	36
160	96	64	32	96	64	64	96
96	64	160	96	64	32	96	32
avg	224/4 = 56	avg	160/4 = 40	avg	136/4 = 34	avg	168/4 = 42

SSTF from 120: 124 is 4 away (nearest), then from 124: 96 (28) beats 160 (36); from 96: 64 (32); last 160 (96). SCAN (LOOK) goes up to 160, then reverses down to 96 and 64. C-SCAN (LOOK) serves 124, 160 going up, jumps from 160 to the lowest request 64 (96 tracks, counted), then goes up to 96.



Arm movement for EO3 2024 question 5. Filled dots: requests served. Time runs downwards.

For the same requests, the no-LOOK versions give SCAN (no LOOK) 120 → 124 → 160 → 199 → 96 → 64: total 214, average 53.50; C-SCAN (no LOOK) 120 → 124 → 160 → 199 → 0 → 64 → 96: total 374, average 93.50.

Other past-paper and tutorial answers (track (tracks traversed))

Policy	Answer
EO3 2023, question 4: 150 tracks, moved 49 → 50 (up); requests 16, 64, 24, 140	
FIFO	16 (34), 64 (48), 24 (40), 140 (116); total 238, average 59.50
SSTF	64 (14), 24 (40), 16 (8), 140 (124); total 186, average 46.50
SCAN (with LOOK)	64 (14), 140 (76), 24 (116), 16 (8); total 214, average 53.50
C-SCAN (no LOOK)	64 (14), 140 (76), 16 (174), 24 (8); total 272, average 68. The row for 16 is $9 + 149 + 16$: up to 149, back to 0, up to 16. Writing the two extra legs as separate rows gives the same total.
Tutorial 9, question 1: 250 cylinders, at 110, previous 135 (down); requests 41, 19, 84, 121, 66, 148	
FCFS	total 330
SSTF	total 167: 110 → 121 → 148 → 84 → 66 → 41 → 19
SCAN (no LOOK)	total 258: 110 → 84 → 66 → 41 → 19 → 0 → 121 → 148
SCAN (LOOK)	total 220
C-SCAN (no LOOK)	total 487: 110 → 84 → 66 → 41 → 19 → 0 → 249 → 148 → 121
C-SCAN (LOOK)	total 247: 110 → 84 → 66 → 41 → 19 → 148 → 121
Mock test, question 1: 160 tracks, moved 101 → 100 (down); requests 80, 22, 113, 60, 147	
FIFO	80 (20), 22 (58), 113 (91), 60 (53), 147 (87); total 309, average 61.80
SSTF	113 (13), 80 (33), 60 (20), 22 (38), 147 (125); total 229, average 45.80
SCAN (no LOOK)	80 (20), 60 (20), 22 (38), 113 (135), 147 (34); total 247, average 49.40
C-SCAN (LOOK)	80 (20), 60 (20), 22 (38), 147 (125), 113 (34); total 237, average 47.40

Properties of the policies

"Selection by requestor" means the choice depends on which process made the request. "Selection by requested item" means it depends on the track asked for.

Policy	Remark	Selection by
FIFO	treats every request in arrival order; no starvation; close to random when many processes compete	requestor
Priority (PRI)	control is outside disk-queue management; low priority may starve	requestor
LIFO (last in, first out: the newest request first)	maximises locality (the next request is often near the last one) and resource use; may starve	requestor
SSTF	high utilisation, small queues; may starve far requests	requested item
SCAN	better service distribution; favours requests near the innermost and outermost tracks, and the latest arrivals	requested item
C-SCAN	lower service variability	requested item
N-step-SCAN	service guarantee. The queue is split into subqueues of N requests (N here is a queue length); each subqueue is served with SCAN. $N = 1$ is FIFO	requested item
FSCAN	load sensitive. Two subqueues: one is served with SCAN while new requests wait in the other	requested item

RAID

Level	Category	Description	Disks for D disks of data
0	striping	non-redundant: data striped over all disks	D
1	mirroring	every disk has a mirror; no write penalty	$2D$
2	parallel access	redundancy by Hamming code; bit-level striping	$D + m$
3	parallel access	bit-interleaved parity on one parity disk	$D + 1$
4	independent access	block-interleaved parity on one parity disk (bottleneck)	$D + 1$
5	independent access	block-interleaved distributed parity	$D + 1$
6	independent access	two independent parity calculations (P and Q), distributed	$D + 2$

D = number of disks that hold data; m = number of check disks for the Hamming code (it grows with D). All levels: the disks act as one logical drive, and data is striped across them; levels 1 to 6 store redundant data so a failed disk can be rebuilt (0 has none). A **strip** is the part of a stripe on one disk; a **stripe** is one strip on every disk. **XOR** ($\oplus$) works on one bit column at a time: the result bit is 1 if the bits differ and 0 if they are the same ($0 \oplus 0 = 0$, $0 \oplus 1 = 1$, $1 \oplus 0 = 1$, $1 \oplus 1 = 0$). For more than two strips, the result bit is 1 when the column has an odd number of 1s.

1. Usable capacity, with n = total number of disks in the array: RAID 5 $(n-1)\times$ disk size; RAID 6 $(n-2)\times$; RAID 1 $n/2\times$.
2. Parity P = XOR of the data strips of a stripe. Rebuild a lost strip = XOR of every surviving strip, parity included.
3. RAID 5 layout (disks and stripes numbered from 0): parity of stripe 0 on the last disk, then one disk to the left per stripe; data fill the other disks left to right.
4. Find a block: stripe = block number ÷ data strips per stripe, rounded down; then count along that stripe's data disks, skipping the parity disk.
5. Small write (RAID 4 and 5): 2 reads (old data, old parity) and 2 writes (new data, new parity = old parity $\oplus$ old data $\oplus$ new data). RAID 6: 3 reads and 3 writes. *(Assumed)*

	disk 0	disk 1	disk 2	disk 3	disk 4
stripe 0	b0	b1	b2	b3	P(0-3)
stripe 1	b4	b5	b6	P(4-7)	b7
stripe 2	b8	b9	P(8-11)	b10	b11
stripe 3	b12	P(12-15)	b13	b14	b15
stripe 4	P(16-19)	b16	b17	b18	b19

RAID 5 with 5 disks (disk 0 on the left).

Exam question: Mock test, question 3

5 disks of 4 TB, so $n = 5$.

- (a) RAID 5: $(5 - 1) \times 4 = 16$ TB. RAID 6: $(5 - 2) \times 4 = 12$ TB.
- (b) 4 data strips per stripe: stripe $= 11 \div 4 = 2.75$, rounded down $= 2$. Stripe 2 holds b8, b9, P, b10, b11 on disks 0 to 4 (parity on disk 2). Block 11: **stripe 2, disk 4**.
- (c) The data strips are 0110, 1011, 1100, 0011 and the parity is 0010 (check: $0110 \oplus 1011 \oplus 1100 \oplus 0011 = 0010$). The disk with 1011 fails. XOR the survivors. First bit column: 0, 1, 0, 0 has one 1, so the bit is 1. All columns: $0110 \oplus 1100 \oplus 0011 \oplus 0010 =$ **1011**, the lost strip.
- (d) Small write: **2 reads and 2 writes**. For example, if 0110 becomes 0000: read 0110 and the parity 0010; new parity $= 0010 \oplus 0110 \oplus 0000 = 0100$; write 0000 and 0100.

Disk cache

A disk cache is a buffer in main memory for disk sectors. A request for a sector in the cache is a **hit**; otherwise a **miss**, and the sector is read into the cache, replacing another.

- **LRU**: replace the block unused for the longest time (a stack, most recently used on top).
- **LFU**: replace the block with the fewest references (a counter per block). Problem: a short burst of references gives a high count to a block that may not be used again.
- **Frequency-based replacement**: a "new section" at the top of the stack; a re-reference there does not raise the count, so a burst counts once.

1. Write the cache contents after every reference, most recently used first.
2. Hit: move the block to the top (LFU: count $+1$). Miss with a full cache: remove the LRU block (LFU: the smallest count; tie: the least recently used *(Assumed)*).

Exam question: Mock test, question 5

A cache of 3 blocks starts empty. References: 5, 3, 5, 2, 4, 3, 1, 5. Give the replaced blocks and the hits for LRU and for LFU.

LRU

Ref	Hit?	Replaced	Cache, most recent first
5	miss		5
3	miss		3, 5
5	hit		5, 3
2	miss		2, 5, 3
4	miss	3	4, 2, 5
3	miss	5	3, 4, 2
1	miss	2	1, 3, 4
5	miss	4	5, 1, 3

Replaced: 3, 5, 2, 4. Hits: **1**.

LFU (count in brackets; tie: least recently used)

Ref	Hit?	Replaced	Cache, most recent first
5	miss		5 (1)
3	miss		3 (1), 5 (1)
5	hit		5 (2), 3 (1)
2	miss		2 (1), 5 (2), 3 (1)
4	miss	3	4 (1), 2 (1), 5 (2)
3	miss	2	3 (1), 4 (1), 5 (2)
1	miss	4	1 (1), 3 (1), 5 (2)
5	hit		5 (3), 1 (1), 3 (1)

Replaced: 3, 2, 4. Hits: **2**. Block 5 has count 2 after the third reference, so it stays while the count-1 blocks are replaced.

3. Chapter 12: File Management

File operations; file organisations; record blocking; B-trees; contiguous, chained and indexed allocation; compaction; free space; UNIX inodes; directories and access rights

Operations on files

Operation	What it does
Create	a new file is defined and placed in the file structure (directory)
Delete	a file is removed from the file structure and destroyed
Open	a process declares an existing file "opened", so it may perform functions on it
Close	the file is closed for that process; it may no longer use the file until it opens it again
Read	a process reads all or part of the data in the file
Write	a process updates the file: adds new data (the file grows) or changes existing data

Exam question: EO3 2024, question 6

"Name and briefly explain the six operations on files that are typically possible in any operating system's file system" (3 marks): **create, delete, open, close, read, write**, each with the explanation above.

Common mistake

"Append" is part of write and "remove" is the same as delete: a list with these instead of open and close lost a mark in 2024.

A file system also keeps **attributes** of each file: owner, creation time, time last modified, access privileges. Terms: **field** (one value) → **record** (related fields) → **file** (related records) → **database** (related data).

File organisations

Organisation	Structure	Access	Suits
Pile	records in order of arrival; variable fields and lengths; no structure	exhaustive search of the whole file	data collected before processing, or hard to organise
Sequential	fixed-format records, same fields in the same order; sorted by a key field	read in key order from the start; new records go to a log (transaction) file, merged later	batch processing of all records in order
Indexed sequential	sequential file + an index on the key + an overflow file	index finds the area, then a short sequential search	both batch runs and single-record lookups
Indexed	no sorted order; several indexes, one per field of interest; variable records	through an index: **exhaustive** (an entry for every record) or **partial** (only records that contain the field)	timely access to single records by several attributes (airline reservations, inventory)
Direct (hashed)	fixed-length records at an address computed from the key (hashing)	direct to the block that holds the record	fast access to one record at a time (directories, pricing tables, schedules)

1. Ask how the file is used: whole file in one order, or one record at a time by key?
2. Whole file, read from start to end, no key lookups ⇒ sequential (records kept in arrival order with different fields ⇒ pile).
3. Single records by key: one key and also batch runs ⇒ indexed sequential; several keys ⇒ indexed; speed for one key with fixed records ⇒ hashed.
4. Give the reason in one line from the table.

Exam question: EO3 2023, question 5: the organisation of JPG and MP3 files, and why

(a) JPG: **sequential**. The image is one compressed stream of segments that is decoded in a fixed order from start to end; no record is looked up by a key. (b) MP3: **sequential**. The audio is a sequence of frames that a player reads and plays in order (which also allows streaming). No official answer exists for this question. *(Assumed)* Lecture homework asks the same for DOC and XLS files.

Mock test, question 1: (a) a bank processing all accounts in account-number order: **sequential** (sorted by key, batch run). (b) sensor readings stored in arrival order with different fields: **pile**. (c) exhaustive index: an entry for every record; partial index: entries only for records that contain the field.

Record blocking

- **Fixed blocking**: fixed-length records; a whole number of records per block; unused space at the end of each block (internal fragmentation).
- **Variable-length spanned**: variable records packed with no unused space; a record may continue in the next block (a pointer links them).
- **Variable-length unspanned**: variable records, no spanning; space is wasted when the next record does not fit.

Blocks are the unit of I/O with secondary storage. Larger blocks: more records per I/O (good for sequential access) but larger buffers and more unused data moved for random access.

Symbol	Meaning (unit)
n	number of records
R	record size (bytes); not the response ratio of Chapter 9
B	block size (bytes); the gap G sits between blocks, not inside B
G	inter-block gap (bytes)
bf	blocking factor: records per block

$$\text{bf} = \left\lfloor \frac{B}{R} \right\rfloor \qquad \text{blocks} = \left\lceil \frac{n}{\text{bf}} \right\rceil$$

$\lfloor\ \rfloor$ means round down; $\lceil\ \rceil$ means round up.

$$\text{file size} = \text{blocks} \times B + (\text{blocks} - 1) \times G$$

$$\text{Unused per full block} = B - \text{bf} \times R.$$

1. bf: divide B by R and round **down** (unspanned: a record may not be split).
2. Blocks: divide n by bf and round **up** (the last, partly filled block is still a block).
3. File size: blocks $\times B$ plus one gap **between** each pair of blocks (blocks $-$ 1 gaps).

Exam question: Tutorial 10, question 1

20,000 records of 208 bytes; 4096-byte blocks; 256-byte gap; unspanned. (a) bf $= \lfloor 4096/208 \rfloor = \lfloor 19.69 \rfloor =$ **19**. (b) blocks $= \lceil 20000/19 \rceil = \lceil 1052.63 \rceil =$ **1,053**. (c) $1{,}053 \times 4096 = 4{,}313{,}088$; $1{,}052 \times 256 = 269{,}312$; total **4,582,400 bytes**. Unused per full block 144 bytes; the last block holds 12 records.

Question	Working and answer
Tutorial 10, second version: 15,000 records of 224 bytes, 4096-byte blocks, 256-byte gap	bf $= \lfloor 18.29 \rfloor = 18$; blocks $= \lceil 833.33 \rceil = 834$; size $3{,}416{,}064 + 213{,}248 = 3{,}629{,}312$ bytes
Mock test, question 4: 25,000 records of 120 bytes, 2048-byte blocks, 64-byte gap	bf 17; blocks 1,471; size $3{,}012{,}608 + 94{,}080 = 3{,}106{,}688$ bytes; 8 bytes unused per full block (internal fragmentation)
Stallings Problem 12.1: 20,000 records of 140 bytes, 3096-byte blocks, 512-byte gap	bf 22; blocks 910; size 3,282,768 bytes

B-trees

B-tree of minimum degree d

- Every node has at most $2d - 1$ keys and at most $2d$ children.
- Every node except the root has at least $d - 1$ keys (at least d children if it is not a leaf); the root has at least 1 key.
- A non-leaf node with k keys has $k + 1$ children. All leaves are on the same level.
- Keys in a node are in increasing order; the subtree between two keys holds only the values between them.
- $d = 2$: 1 to 3 keys per node. $d = 3$: 2 to 5. $d = 4$: 3 to 7.

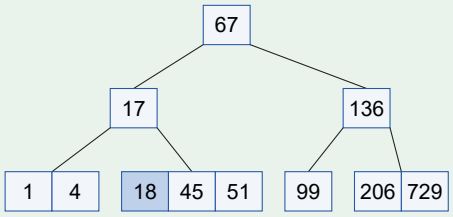
Splitting rule for exam answers: pre-emptive splitting (Tutorial 10 solution: “For the Exam use pre-emptive splitting”). The median of a full node of $2d - 1$ keys is its d-th key.

1. If the root is full ($2d - 1$ keys), split it first: its median becomes the only key of a new root, and the two halves ($d - 1$ keys each) become its children. The tree grows one level.
2. Walk down from the root towards the place of the new key. Before you enter a child that is full, split it: its median moves up into the current node, and its halves become two children.
3. At the leaf, insert the key in sorted order. (A leaf may end up full; it is split only when a later key must pass through it.)
4. Draw each node as a row of boxes with a line from each gap between keys (and each end) to a child. Check every node against the rules.

Exam question: EO3 2024, question 7

Insert 17, 136, 729, 4, 51, 206, 99, 67, 1, 45, 18 in this order into a valid B-tree of degree $d = 2$. Draw the final tree. ($d = 2$: at most 3 keys per node; a full node splits around its 2nd key.)

Insert	What happens	Tree after (root first, one level per group)
17	17 goes into the leaf which is now [17]	[17]
136	136 goes into the leaf which is now [17 136]	[17 136]
729	729 goes into the leaf which is now [17 136 729]	[17 136 729]
4	the root is full so it is split first and the tree grows one level; split [17 136 729] around its middle key 136: [17] | [729]; 136 moves up; 4 goes into the leaf which is now [4 17]	[136] / [4 17] [729]
51	51 goes into the leaf which is now [4 17 51]	[136] / [4 17 51] [729]
206	206 goes into the leaf which is now [206 729]	[136] / [4 17 51] [206 729]
99	split [4 17 51] around its middle key 17: [4] | [51]; 17 moves up; 99 goes into the leaf which is now [51 99]	[17 136] / [4] [51 99] [206 729]
67	67 goes into the leaf which is now [51 67 99]	[17 136] / [4] [51 67 99] [206 729]
1	1 goes into the leaf which is now [1 4]	[17 136] / [1 4] [51 67 99] [206 729]
45	split [51 67 99] around its middle key 67: [51] | [99]; 67 moves up; 45 goes into the leaf which is now [45 51]	[17 67 136] / [1 4] [45 51] [99] [206 729]
18	the root is full so it is split first and the tree grows one level; split [17 67 136] around its middle key 67: [17] | [136]; 67 moves up; 18 goes into the leaf which is now [18 45 51]	[67] / [17] [136] / [1 4] [18 45 51] [99] [206 729]



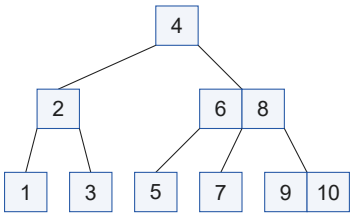
Final tree: [67] / [17] [136] / [1 4] [18 45 51] [99] [206 729].

Common mistake

Other splitting rules give other trees. Use the pre-emptive answer above.

- Insert the key, then split any node that has too many keys (the left middle key moves up): [17 51 136] / [1 4] [18 45] [67 99] [206 729].
- Split a full leaf only when a key must enter it, and never split the root first: [17 67 136] / [1 4] [18 45 51] [99] [206 729]. A 2024 script with this tree received 1.5 of 3 marks; the marking does not say why.

EO3 2023, question 6: start [2] / [1] [3 4], $d = 2$, insert 5 to 10.
Final: [4] / [2] [6 8] / [1] [3] [5] [7] [9 10].



EO3 2023 question 6, final tree.

Question	Final tree (root first, one level per group)
Tutorial 10, question 2: $d = 4$, keys 5, 38, 27, 33, 29, 44, 47, 12, 48, 54, 50, 62, 8, 65, 58, 70, 74, 1, 92, 81	[33 48 62] / [1 5 8 12 27 29] [38 44 47] [50 54 58] [65 70 74 81 92] After inserting 76: [33 48 62] / [1 5 8 12 27 29] [38 44 47] [50 54 58] [65 70 74 76 81 92]
Tutorial 10, second key set: $d = 4$, keys 2, 41, 23, …, 85	[32 45 59] / [2 3 6 14 23 30] [41 42 43] [51 52 57] [61 68 71 85 89] After inserting 79: [32 45 59] / [2 3 6 14 23 30] [41 42 43] [51 52 57] [61 68 71 79 85 89]
Mock test, question 2: $d = 3$	[25 55] / [5 10 15 20] [30 40 50] [60 65 70 85] After 75 and 80: [25 55 70] / [5 10 15 20] [30 40 50] [60 65] [75 80 85]
Mock test, question 3: $d = 2$, start [20] / [5 10] [30 40], insert 50 to 100	[40] / [20] [60 80] / [5 10] [30] [50] [70] [90 100]

File allocation

The **file allocation table (FAT)** records, for each file, which blocks it occupies. A **portion** is a run of blocks given to a file in one allocation.

	Contiguous	Chained	Indexed
FAT entry	start block, length	start block, length; each block holds a pointer to the next	the index block (it lists the blocks, or (start, length) portions)
Preallocation	necessary	possible	possible
Portions	variable size, large	fixed blocks, small	fixed blocks, or variable portions
Allocation frequency	once	low to high	low to high
Time to allocate	medium	long	short to medium
Main problem	external fragmentation; needs compaction	no direct access; must follow the chain; consolidation helps locality	index block overhead

Consolidation: moving a chained file's blocks next to one another. **Locality**: blocks that are used together lie close together on the disk, so the arm moves less. **Preallocation**: the maximum size is fixed when the file is created. **Dynamic allocation**: space is given in portions as needed. Contiguous allocation gives good sequential and direct access (block i is at start $+i$).

Steps for contiguous allocation and compaction

1. Grow by k: if the k blocks right after the file are free, the length grows by k and the start stays. (If not, the file must move to a free run that fits, or the disk must be compacted first. *(Assumed)*)
2. Shrink by k: the length drops by k; the start stays; the last k blocks become free.
3. Delete: remove the row; its blocks become free.
4. Compaction: sort the files by start block (their order on the disk); the first starts at block 0; each next start $=$ previous start $+$ previous length. Lengths do not change.

Exam question: EO3 2024, question 8

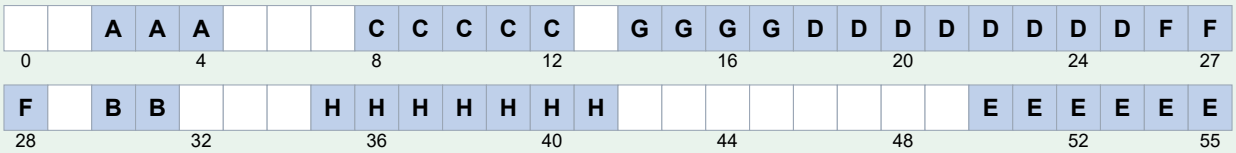
Contiguous allocation, FAT: A 2/3, B 30/2, C 8/5, D 18/8, E 50/6, F 26/3, G 14/4, H 35/7 (start/length). The disk is compacted; give the resulting FAT.

Disk order by start block: A (2), C (8), G (14), D (18), F (26), B (30), H (35), E (50). Packed from 0 (each start $=$ previous start $+$ previous length): A 0, C $0+3=3$, G $3+5=8$, D $8+4=12$, F $12+8=20$, B $20+3=23$, H $23+2=25$, E $25+7=32$.

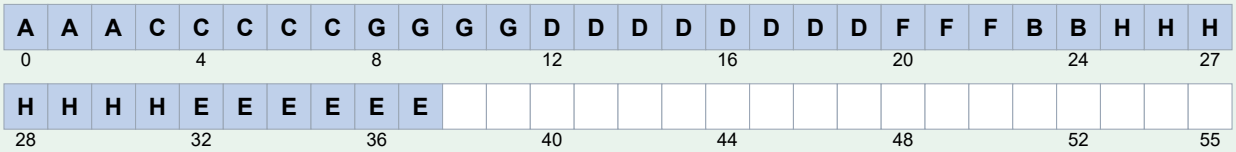
File	A	B	C	D	E	F	G	H
Start	0	23	3	12	32	20	8	25
Length	3	2	5	8	6	3	4	7

Check: the lengths add up to 38 blocks. Packed from block 0, the files fill blocks 0 to $38-1$, so the last file ends at block $32+6-1=37$.

Before compaction



After compaction



Blocks 0 to 55. The files keep their disk order; the free space becomes one run at the end.

Question	After the changes (start/length)	After compaction
EO3 2023, question 7: Homework +1, Photo -2, Backup deleted, ToDo -1	Homework 53/6, Music 60/10, MyMarks 80/3, Photo 30/7, ToDo 94/3	Homework 7/6, Music 13/10, MyMarks 23/3, Photo 0/7, ToDo 26/3
Mock test, question 5	Thesis 14/5, Budget 40/5, Photos 0/10, Songs 50/13	Thesis 10/5, Budget 15/5, Photos 0/10, Songs 20/13

Free space management

- **Bit table**: one bit per block, block 0 first; 0 = free, 1 = in use. Small, but searching it is slow for a large disk.
- **Chained free portions**: free portions linked by a pointer and length in each portion; little overhead, but fragmentation makes allocation slow.
- **Indexing**: free space is treated as a file with an index table of free portions.
- **Free block list**: each free block has a number (24 or 32 bits) in a list kept on disk; part of it is held in memory as a stack or FIFO queue.

$$\text{bit-table bytes} = \frac{\text{disk bytes}}{8 \times \text{block bytes}}$$

1. Write both sizes as powers of 2 (K $= 2^{10}$, M $= 2^{20}$, G $= 2^{30}$, T $= 2^{40}$ bytes). When a question does not say, use these binary units. *(Assumed)*
2. Blocks $=$ disk $\div$ block $=$ number of bits.
3. Bytes $=$ bits $\div 8$.

Exam question: Mock test, question 6

(a) 20 blocks (0 to 19), A 1/3, B 6/2, C 10/4, D 17/2 (start/length). Blocks in use: A blocks 1–3; B blocks 6–7; C blocks 10–13; D blocks 17–18. Write 1 for these blocks and 0 for the rest, block 0 first. In groups of five (blocks 0–4, 5–9, 10–14, 15–19): `01110 01100 11110 00110`, so `01110011001111000110`. (b) 64 GB, 4 KB blocks: $2^{36}/2^{12} = 2^{24} = 16{,}777{,}216$ bits; $\div 8$: $2^{21} = 2{,}097{,}152$ bytes (2 MB).

Stallings example: a 16 GB disk with 512-byte blocks needs $2^{34}/(8 \times 2^9) = 4{,}194{,}304$ bytes (4 MB) of bit table, which itself fills 8,192 blocks.

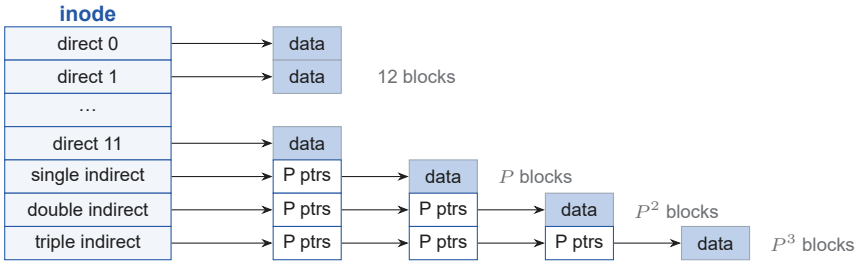
UNIX inodes

An inode holds a file's attributes and 15 block pointers: 12 direct, then one single, one double and one triple indirect pointer.

$$P = \frac{\text{block size}}{\text{pointer size}} \quad \text{pointers per block}$$

Data blocks reachable: direct 12; single P; double P^2; triple P^3. Bytes = blocks $\times$ block size.

1. P = block bytes $\div$ pointer bytes.
2. Logical block of a byte offset $= \lfloor \text{offset}/\text{block size} \rfloor$.
3. Blocks 0 to 11: direct. Next P blocks: single indirect. Next P^2: double. Then triple.



Inode pointers: 12 direct, then single, double and triple indirect; each pointer block holds P pointers.

Exam question: Mock test, question 7

4 KB blocks, 4-byte pointers. (a) $P = 2^{12}/2^2 = 1{,}024$. (b) double indirect: $1{,}024^2 = 1{,}048{,}576$ blocks $= 2^{32} = 4{,}294{,}967{,}296$ bytes (4 GB). (c) offset 1,000,000: $\lfloor 1000000/4096 \rfloor = 244$ (remainder 576); blocks 12 to 1,035 use the single indirect pointer, so **single indirect**, entry $244 - 12 = 232$.

Stallings Table 12.3 (4 KB blocks, 8-byte pointers): $P = 512$; single 512 blocks (2 MB), double 262,144 blocks (1 GB), triple 134,217,728 blocks (512 GB); maximum file 550,831,702,016 bytes.

Directories, access rights and sharing

Directory entry information: basic (file name, type, organisation); address (volume, starting address, size used, size allocated); access control (owner, access information, permitted actions); usage (dates created, last read, last modified, last backup; identities of creator, last reader, last modifier; current usage).

Directory operations: search, create file, delete file, list directory, update directory.

Structures: single list; **two-level** (a master directory with one entry per user, each user directory a simple list; names unique only per user); **tree** (hierarchical: master directory, subdirectories, files). A file is named by its **pathname** from the master directory; the **working directory** allows names relative to it.

Access rights (each usually includes the ones above it):

1. None: the user cannot even see the file.
2. Knowledge: can see that the file exists and who owns it.
3. Execution: can load and run it, not copy it.
4. Reading: can read it for any purpose, including copying.
5. Appending: can add data, not change or delete existing data.
6. Updating: can modify, delete and add data.
7. Changing protection: can change other users' rights.
8. Deletion: can delete the file.

User classes: owner (usually the creator; full rights; may grant rights to others), specific users, user groups, all.
Simultaneous access: lock the whole file or single records; the readers/writers problem applies.